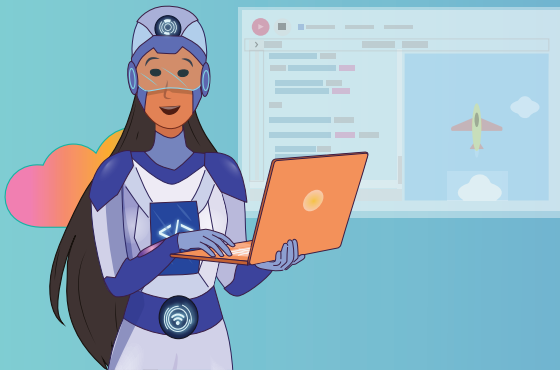
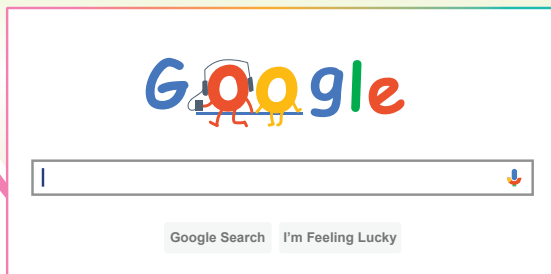
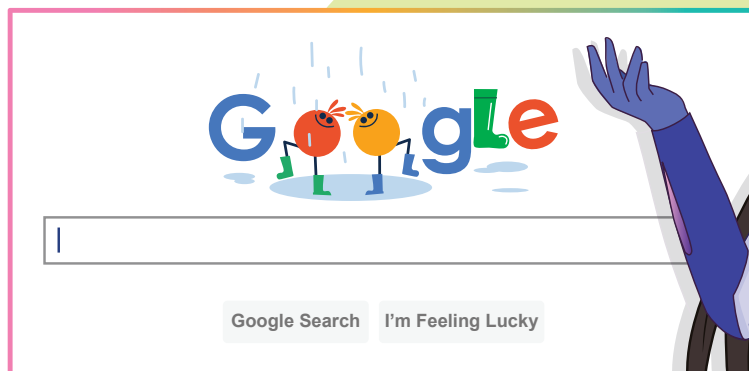


Code to Animate

06



Have you ever seen interactive animations in Google doodles? We can create similar graphics and animations using code.



When was the last time you travelled on a bus or a train? Do you remember looking out of the window and watching trees move backwards?

When we're on a train or a bus, everything outside appears to be moving backwards. But in reality, the train or bus is moving and not the objects outside. This phenomenon is called relative motion and it is the principle computer-generated graphics work on.

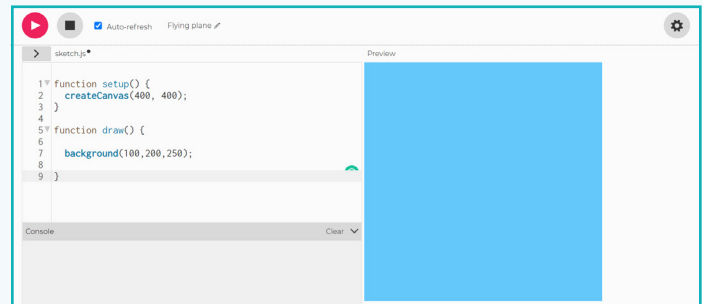
In this lesson, we will use the concepts of variables and conditions to create a code for a moving airplane animation based on the concept of relative motion. We will also learn about flowcharts: a method used to create a logical sequence for a program.

Activity

Create an airplane using various shapes and variables to position objects to make them move.

1 Open p5js editor

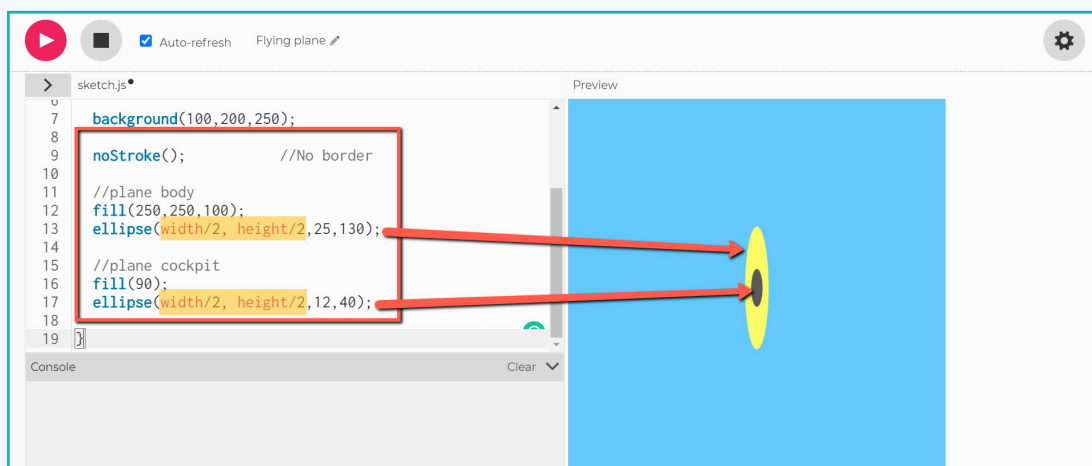
- a. Change the background colour to light blue with $r=100$, $g=200$, $b=250$



We will create an airplane using ellipses, triangles, and rectangles at the centre of the canvas at width/2 and height/2.

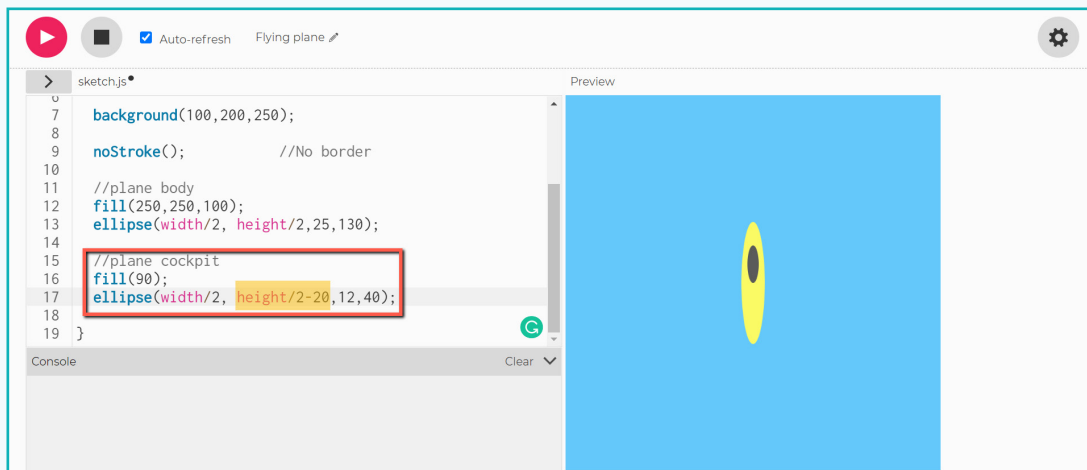
2 Add two ellipses for the body and cockpit of the plane at the centre of the canvas. Use appropriate colours without a border

Use default variables width, height, and division (/) operators to place the ellipse at the centre of the canvas.



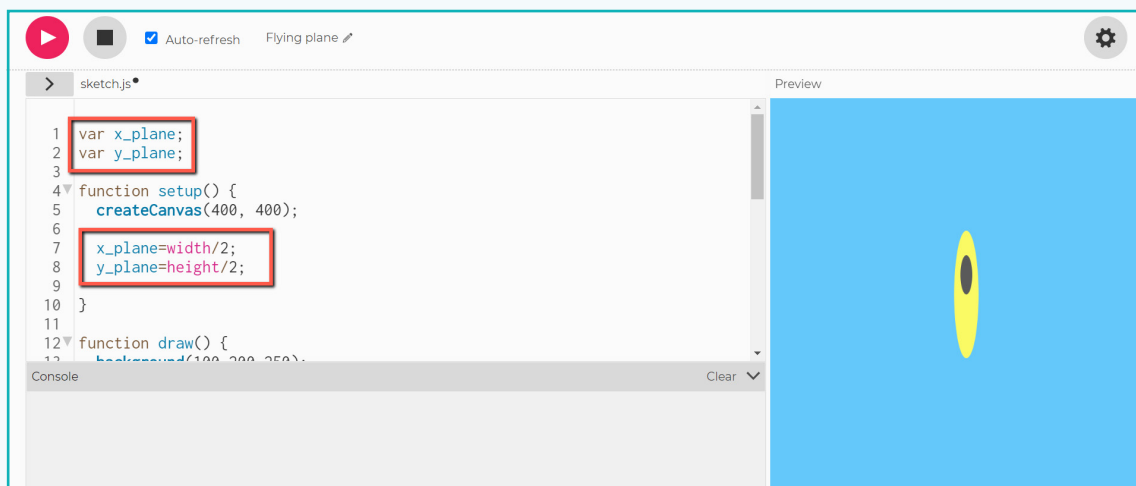
Activity

- 3 To move the cockpit of the plane at an appropriate position, reduce the y position by 20 pixels to move it upwards. Use a subtraction operator



If we want to change an object's position, we need to change the x and y positions of all shapes in it. Currently, it is width/2 and height/2. To avoid this tedious work, we use variables.

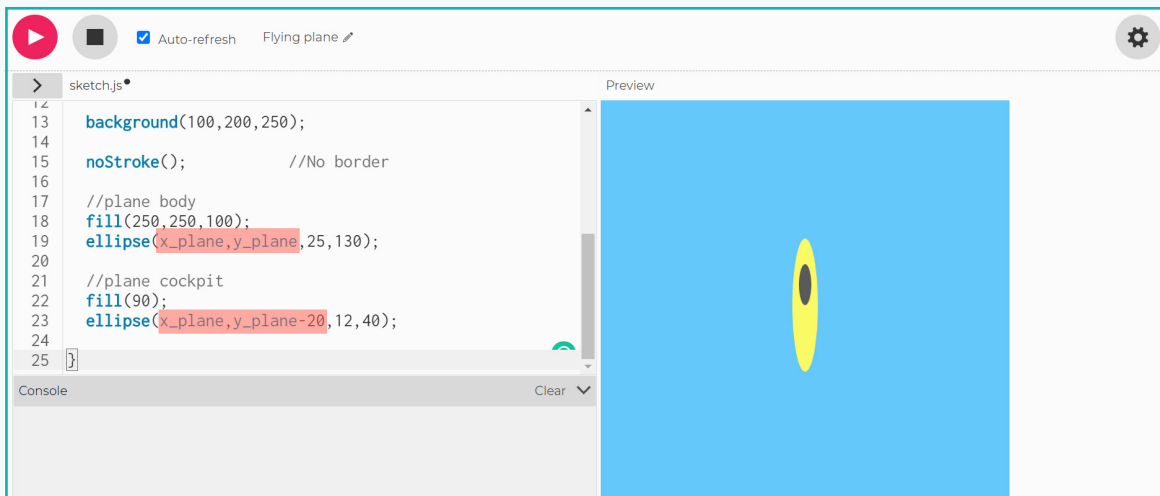
- 4 Create two variables `x_plane` and `y_plane` at the start of the code for x and y locations of the airplane respectively. Set their initial values as `x_plane = width/2` and `y_plane = height/2` in `setup()` function



Activity

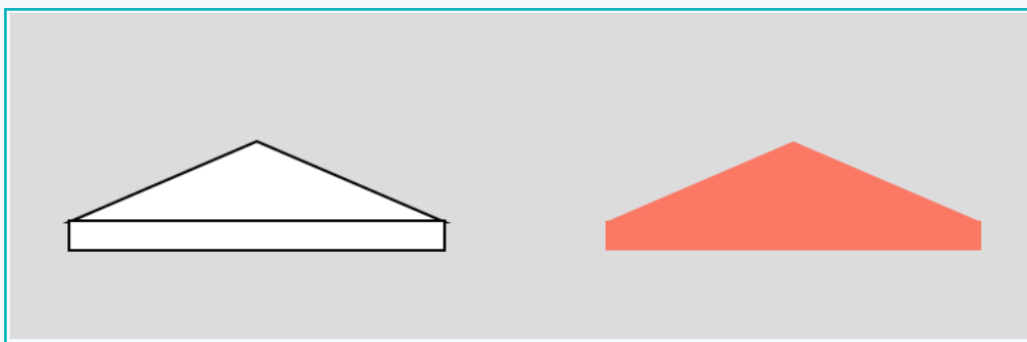
Now we can replace x and y position values in all shapes by variables 'x_plane' and 'y_plane'. If we want to change the position, we only change the values in these variables.

- 5 Update x, y positions (width/2 and height/2) in the shapes drawn. Replace them with variables x_plane and y_plane



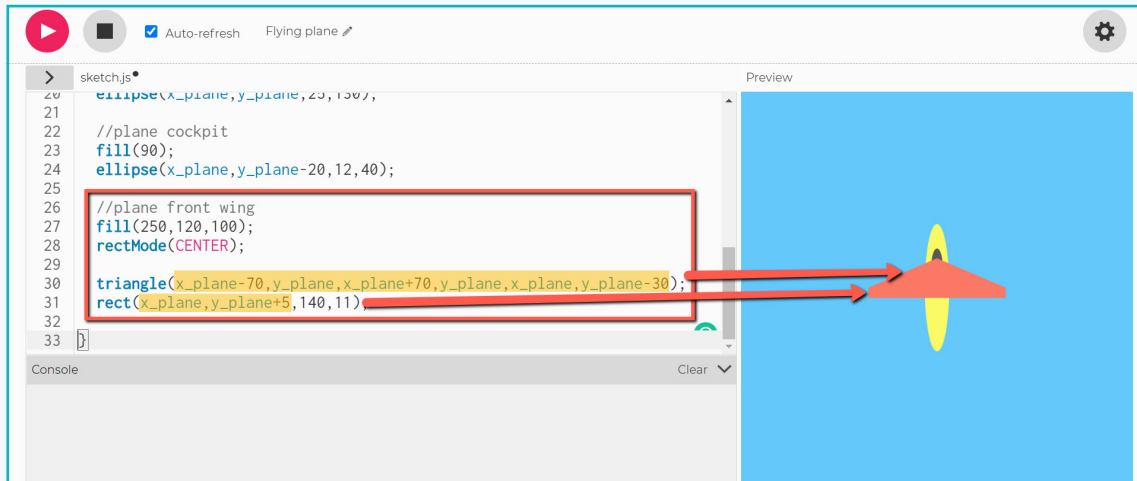
Even after this, the shapes are still in the same position. We will now create the front and back wings of the airplane using the newly created variables to add x and y positions.

- 6 Add the front wing by drawing a coloured triangle and rectangle

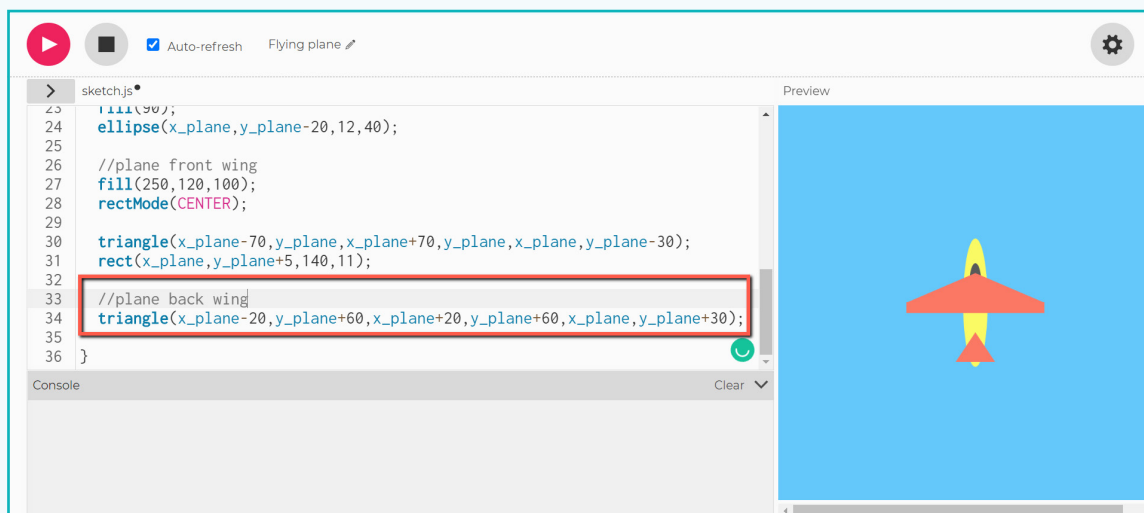


Activity

- Use `rectMode(CENTER)` to correctly place the rectangle.

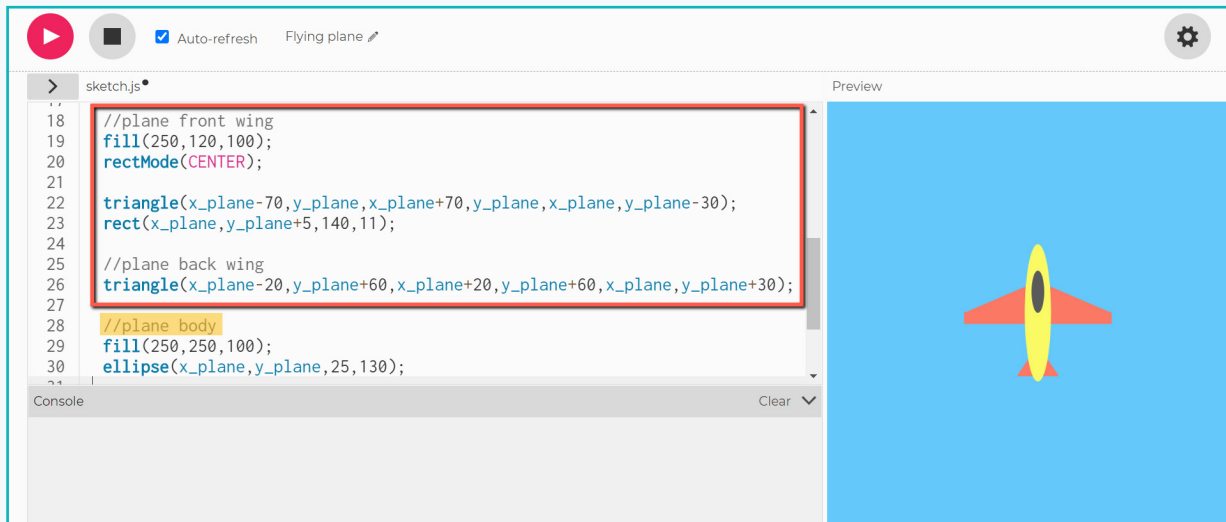


7 Use the same process to add the back wing



Activity

- a. As the airplane body should be over the plane wing, move the code for both wings before the code for the plane's body.



Activity

Side Quest

Create a throttle effect using the 'random' command:

- `random()` - This command provides a random number between the input range to the command.

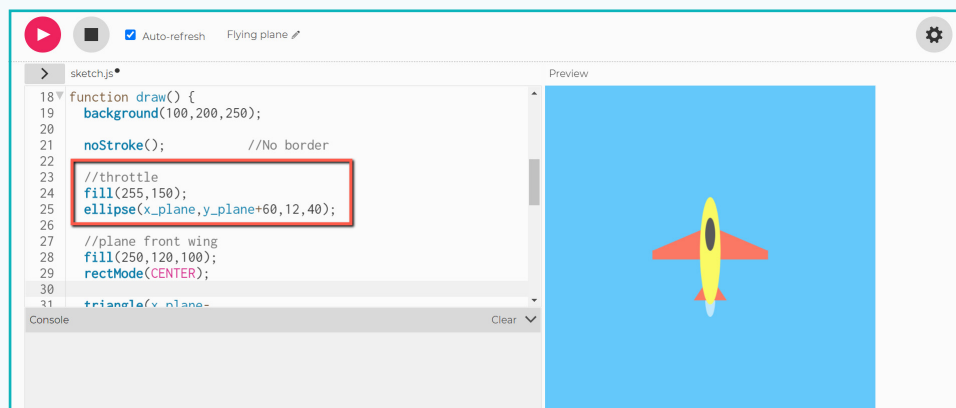
For example:

`random(60)` command gives a random number between 0 and 60.

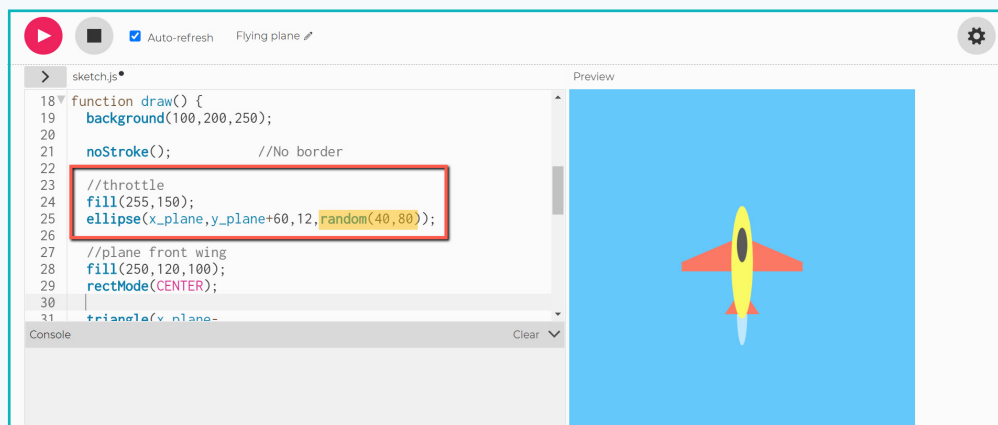
`random(30,60)` command will give a random number between 30 and 60.

We can add an ellipse at the end of the plane and set its length as a random value. It will vary randomly to create an effect resembling a flame-like throttle.

- b. Add a white coloured semi-transparent ellipse(`x_plane, y_plane, 10, 40`); before the code for the plane body.



- c. Instead of height = 40, add `random(40,80)`. This will give it a random height between 40 to 80.

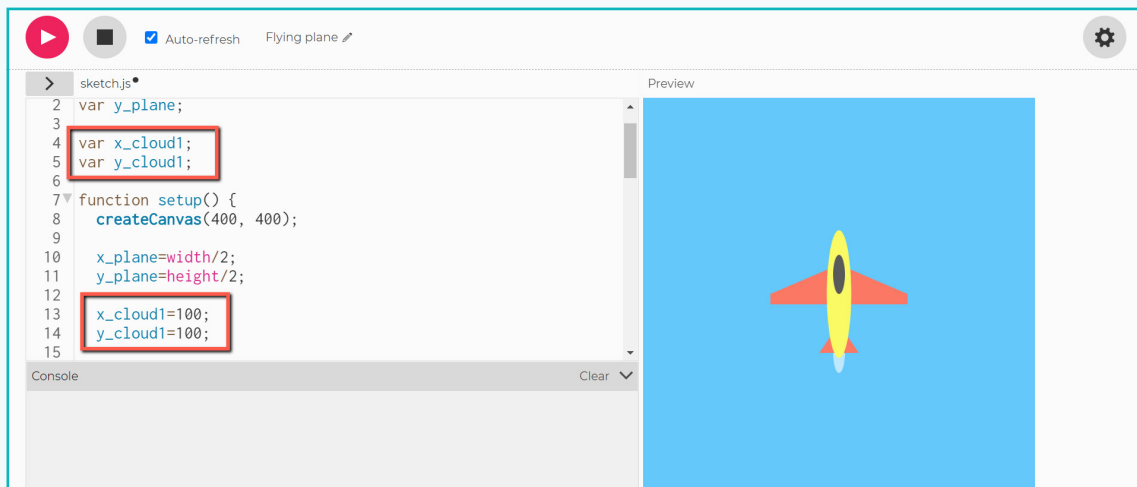


Activity

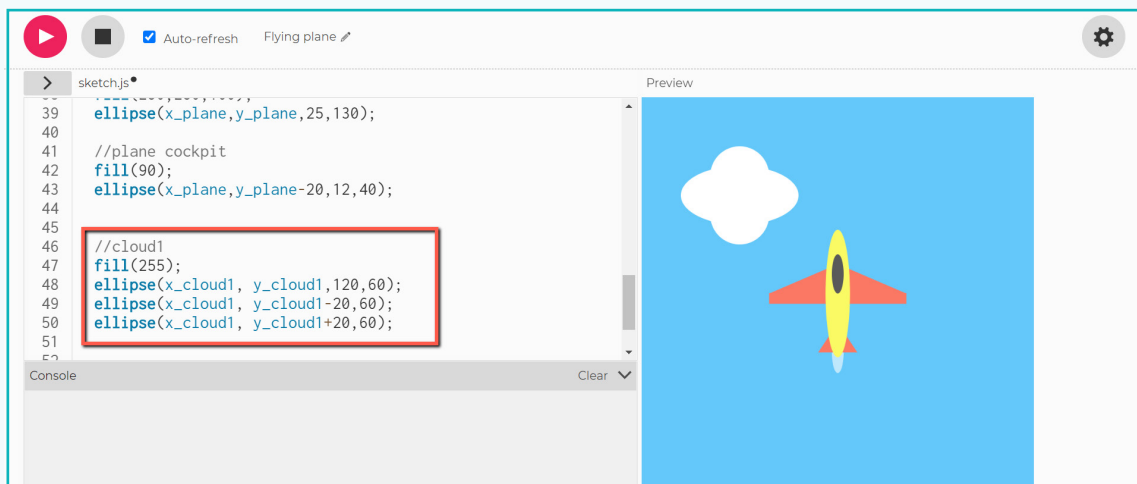
It will appear as a throttle, spitting fire out just like in a flying airplane!

Let's create clouds as surroundings and to show relative plane movement using variables. Can you create them on your own?

- 8 Create variables `cloud1_x` and `cloud1_y` for positioning and initialize their initial values in `setup()`



- 9 Create a cloud using multiple white coloured ellipses overlapping each other



If a plane is flying forward, clouds will appear to move backwards, in the opposite direction. If we move the airplane, it will move out of the screen. Instead, if we move clouds in the opposite direction, the plane will appear to move forward. This is also called as Moving Background Effect.

To update the position of the clouds, we will use increment/decrement operator.

Increment/decrement operators: Repeatedly incrementing or decrementing (increasing or decreasing) the value inside a variable by a certain number is an important operation in programming.

For example: A variable `y_cloud1` is used for the y location of the plane with its initial value as 0. We want to continuously increase its value by 1. It can be done as follows:

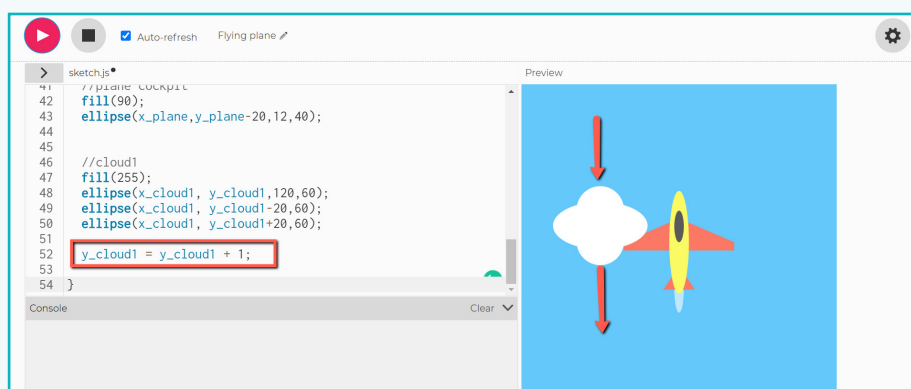
`y_cloud1 = y_cloud1 + 1;`

The process of incrementing, decrementing works like this.

Increment	Decrement
<code>y_cloud1</code> → 0	<code>y_cloud1</code> → 5
<code>y_cloud1 = y_cloud1 + 1;</code>	<code>y_cloud1 = y_cloud1 - 1;</code>
1 0	4 5
<code>y_cloud1</code> → 1	<code>y_cloud1</code> → 4

Activity

- 10** To move the cloud downwards, its y position needs to be increased. As the `draw()` function runs repeatedly, we can increment the value in the `y_cloud1` variable inside the `draw()` function



Activity

Our cloud is moving downwards now! But it disappears after it reaches the bottom edge. So, we will have to create new clouds each time to show continuous stream of clouds.

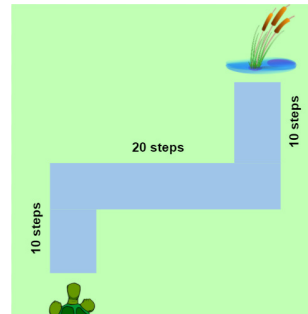
There is a simpler method to do this than creating new clouds every time. We will learn about conditions and how to use them to always keep the cloud on the canvas. This way it will appear as though new clouds are continuously passing by.

There is a simpler method to do this than creating new clouds every time. We will learn about conditions and how to use them to always keep the cloud on the canvas. This way it will appear as though new clouds are continuously passing by.

Exercise

- Output of random(10,20) command cannot be _____.
 - 12 ☐
 - 15 ☐
 - 18 ☐
 - 9 ☐
- If the cursor is at point (100,150) on the canvas, what will be the value stored in the variable loc, if `loc = mouseX/2`?
 - 75 ☐
 - 100 ☐
 - 50 ☐
 - 150 ☐
- If the value stored in the variable `x = 10`, what will be the value in variable `x` after executing the code line `"x = x + 2;"`?
 - 10 ☐
 - 20 ☐
 - 2 ☐
 - 12 ☐

Before we start coding further, can you solve a problem first? Observe the following image and write the steps of all the actions in a sequence, such that the turtle will reach the pond. We will verify the answer soon.

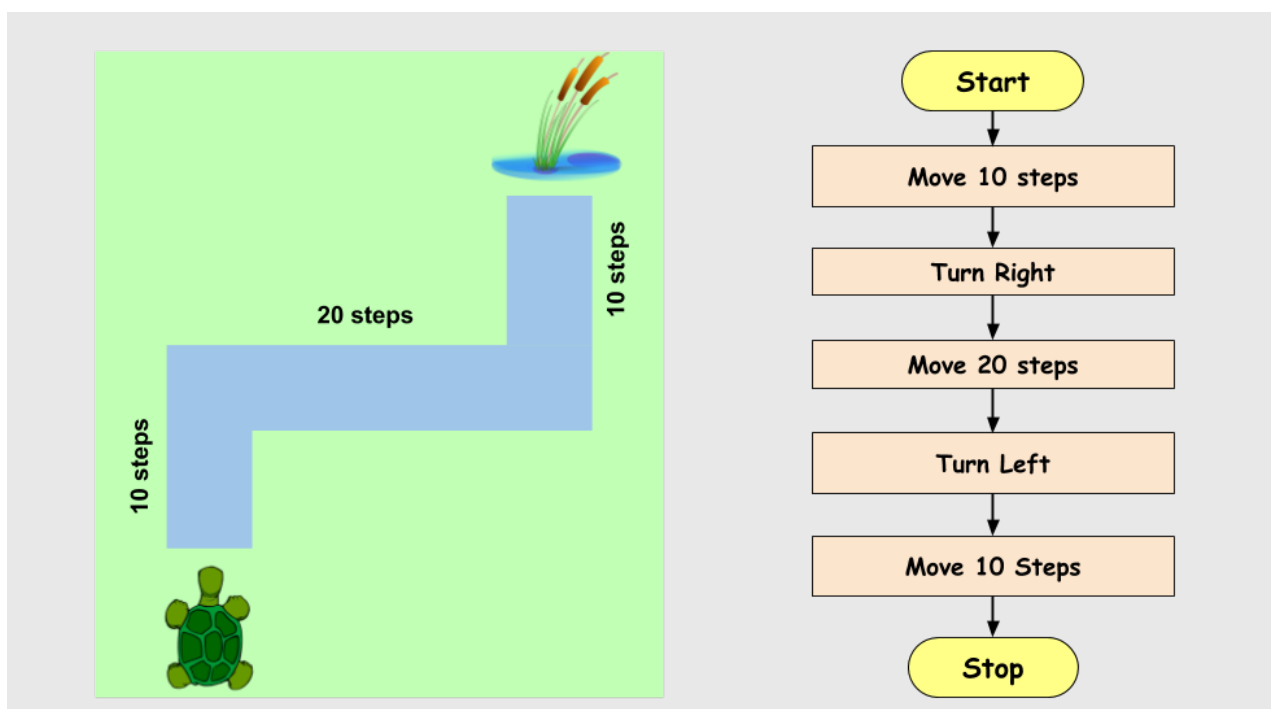


So far, we have written smaller codes. As we learn more, codes become larger and more complex. Planning the logic and flow of the code before writing it helps to improve accuracy and reduces errors. This planning is done using diagrams called flowcharts. Before we start coding, let's learn about flowcharts.

Flowchart

A flowchart is a special type of diagram that shows the logical steps for the instructions in a process, or for a code to achieve its desired output.

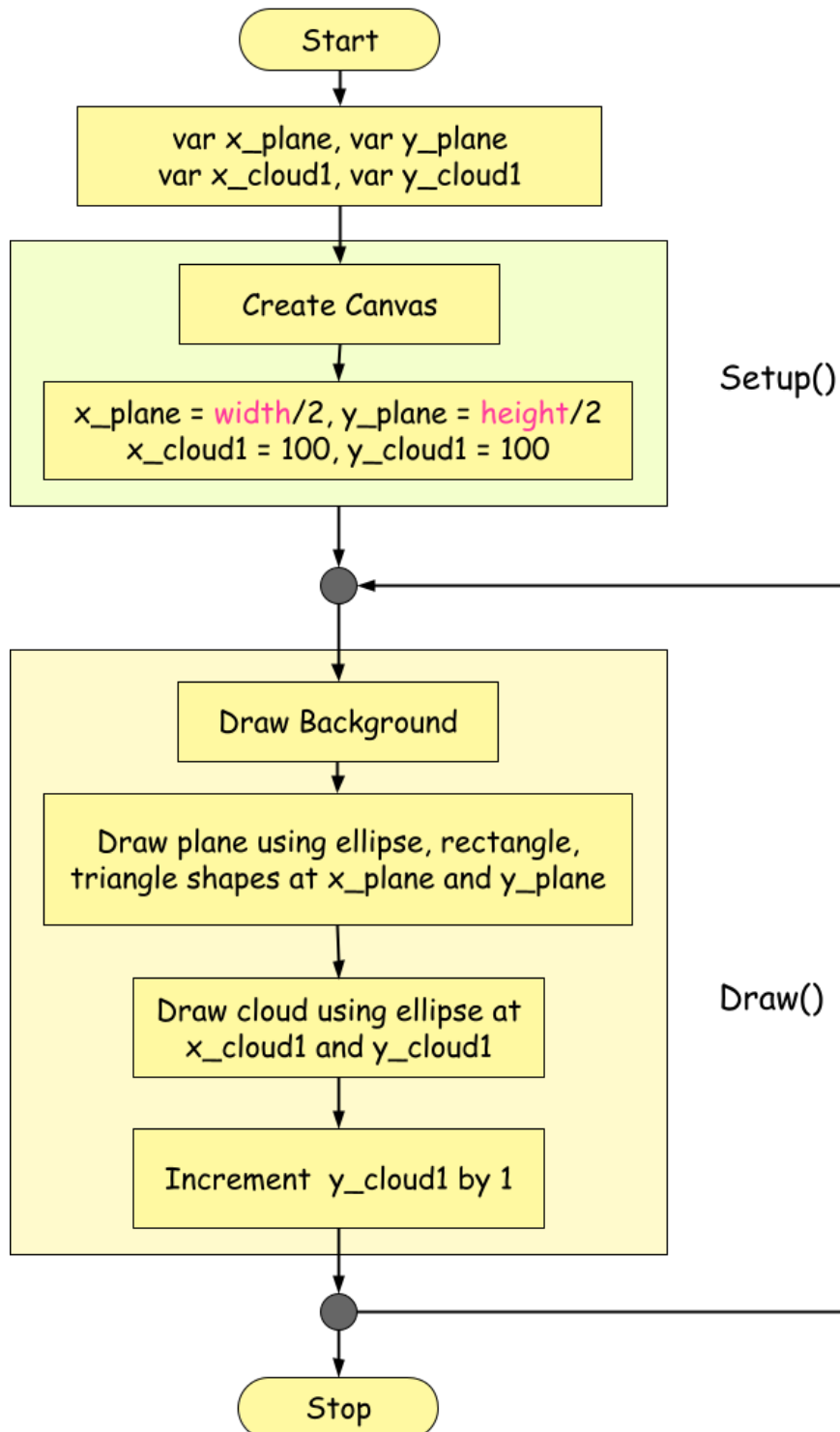
For example: For the turtle to reach the pond, the sequence of actions can be shown through the following flowchart



Different shapes are used to show various actions while drawing a flowchart.

Shape	Name	Description
	Flow line	An arrow shows the direction of the flow of steps.
	Terminal	Used as start and stop blocks as the start and end of the flowchart.
	Process	Used to show the steps in a process.
	Decision	Used to show the output of the condition, depending on whether it is true or false.
	Input / Output	Used to show user input or output display.

Let's make a flowchart to record the code we have created so far, to make the airplane look like it is flying.



Exercise

State whether True or False

4 Flow lines are the arrows connected to different types of boxes to show the direction of flow of the steps.

5 Flowcharts are drawn after writing a code for a specific task.

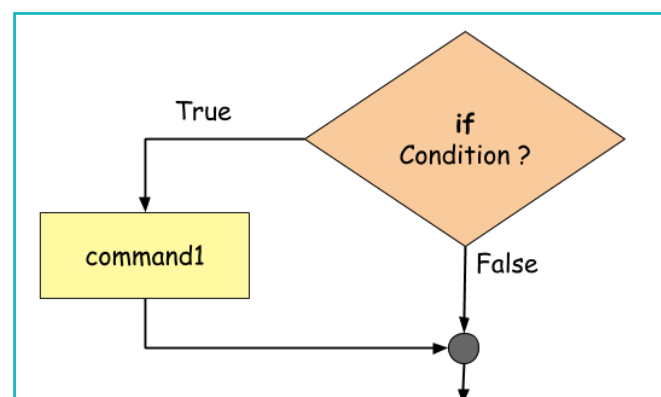
6 Flowcharts help programmers create a step-wise logical process and write the correct code to solve a problem.

In the **if condition**, the 'if' statement executes a snippet of code only if a specified condition is true. The condition is added inside round brackets after the word 'if'. The code to be executed is added in curly brackets.

if (condition) {...}

In the airplane example, we can use an 'if' condition so that the cloud doesn't exit the screen. There are two parts to 'if' conditions.

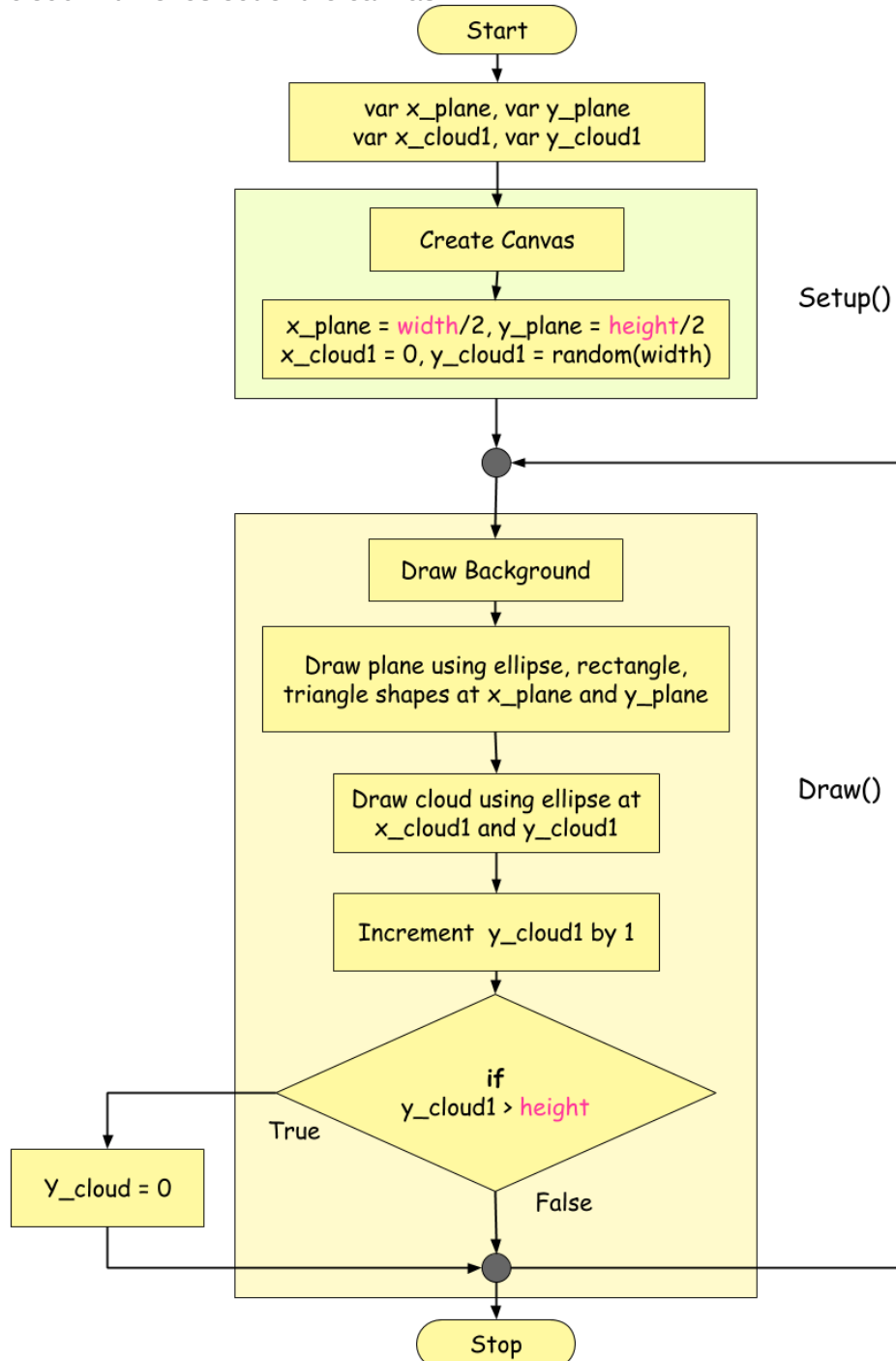
1. Condition: The condition states that the cloud will move out of the bottom edge if its y location is greater than the height of the canvas.
2. Statements to be executed if the condition is true - As a result, we want to move the cloud to the upper edge, so that it can move downwards again. i.e. change 'y' position to 0.



```

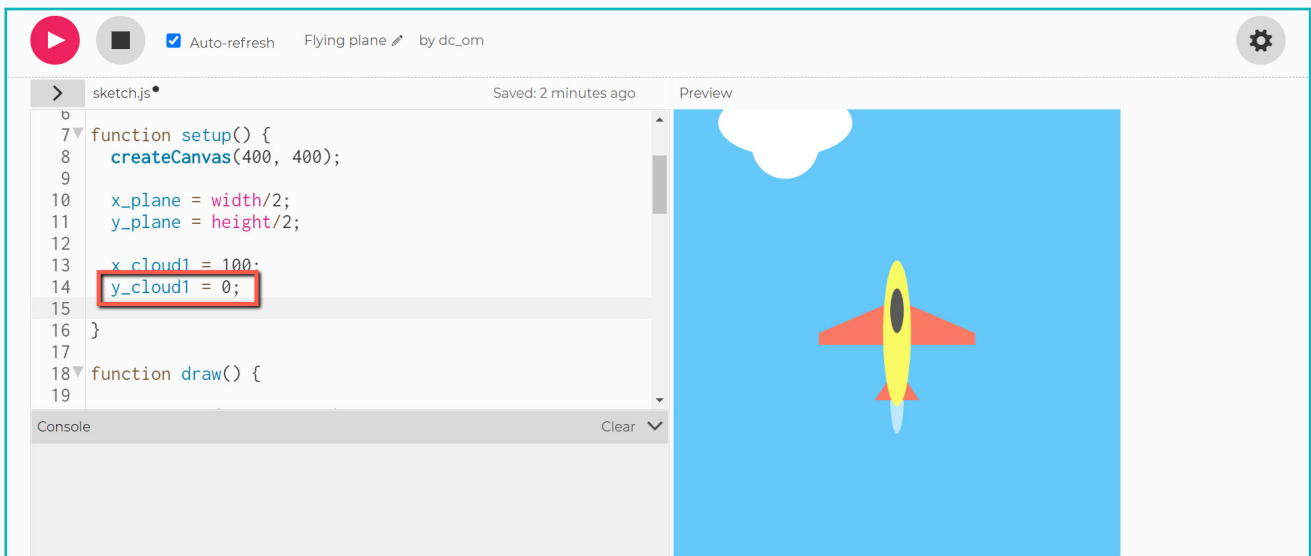
If (y_cloud1 > height) {
    y_cloud1 = 0 ;
}
    
```

We will update the flowchart to change the initial position of the cloud and use an 'if' condition to reposition the cloud if it moves out of the canvas.

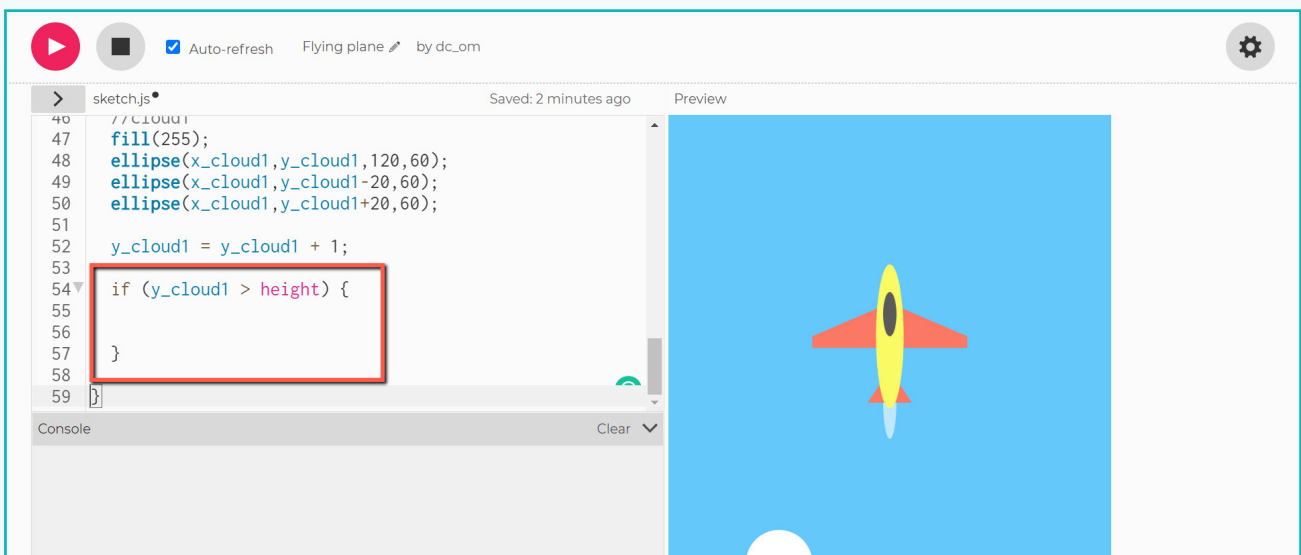


Activity

- 11** To show the cloud coming in from the upper edge, change the initial value of `y_cloud1 = 0` in `setup()`

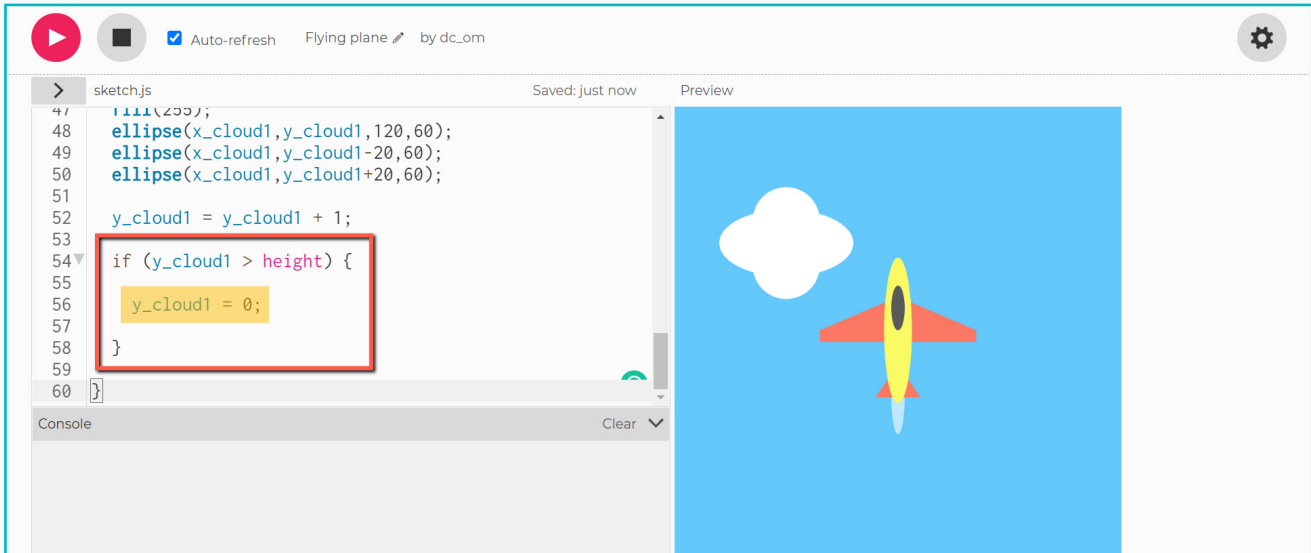


- a. Add 'if' condition in `draw()` command with condition '`y_cloud1 > height`'.



Activity

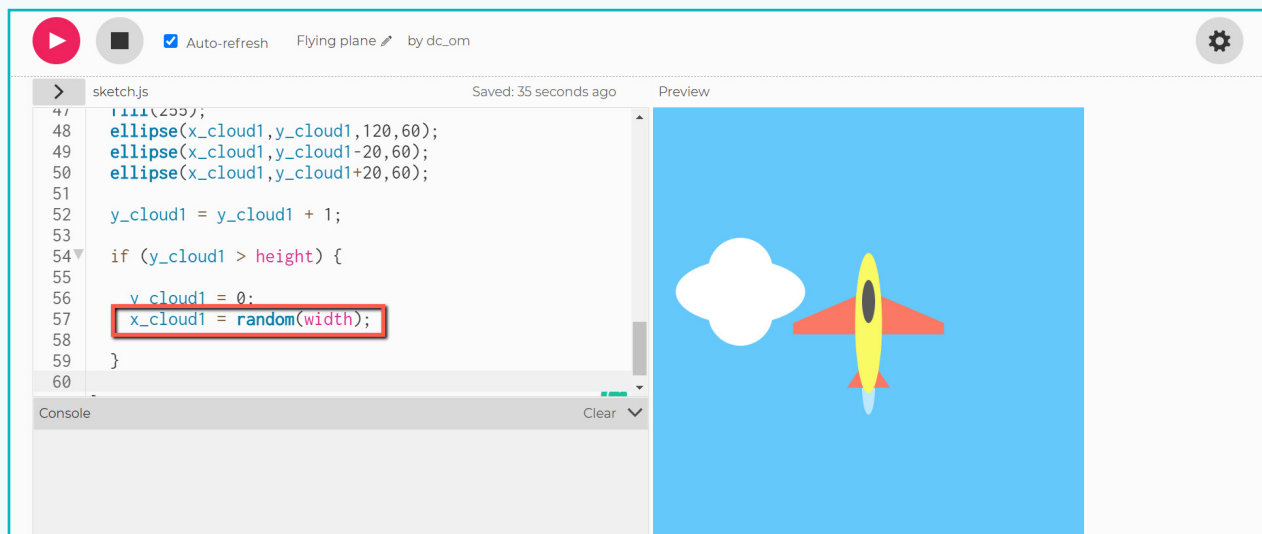
b. If the condition is met, set variable `y_cloud1 = 0`.



Now, the cloud goes out of the screen from the bottom edge and comes in from the top edge.

This feels unnatural and boring since the cloud keeps moving on the same path. To change this, we can change the cloud's x position to a random value, such that the cloud comes out from the top edge from a random spot.

c. Add `x_cloud1 = random(width);` inside the if condition statement.



Activity

Use similar code to add some more clouds that will move simultaneously.

- d. Add a smaller cloud and place its commands before the code for the airplane. This way, the plane will appear to move above the cloud.

```

> sketch.js
3
4 var x_cloud1;
5 var y_cloud1;
6
7 var x_cloud2;
8 var y_cloud2;
9
10 function setup() {
11   createCanvas(400, 400);
12
13   x_plane = width/2;
14   y_plane = height/2;
15
16   x_cloud1 = 100;
17   y_cloud1 = 0;
18
19   x_cloud2 = 300;
20   y_cloud2 = 0;
21
22 }
    
```

```

> sketch.js
24 function draw() {
25   background(100,200,250);
26
27   noStroke();
28
29   //cloud2
30   fill(255);
31   ellipse(x_cloud2,y_cloud2,80,40);
32   ellipse(x_cloud2,y_cloud2-10,40);
33   ellipse(x_cloud2,y_cloud2+10,40);
34
35   //throttle
36   fill(255,150);
37   ellipse(x_plane,y_plane+60,12,random(40,80));
38
39   //plane front wing
40   fill(250,120,100);
41   rectMode(CENTER);
42
    
```

- e. Since cloud2 is farther than cloud1, it will appear to move slower than cloud1. To move it at a slower speed, we will add a lower value to the y position.

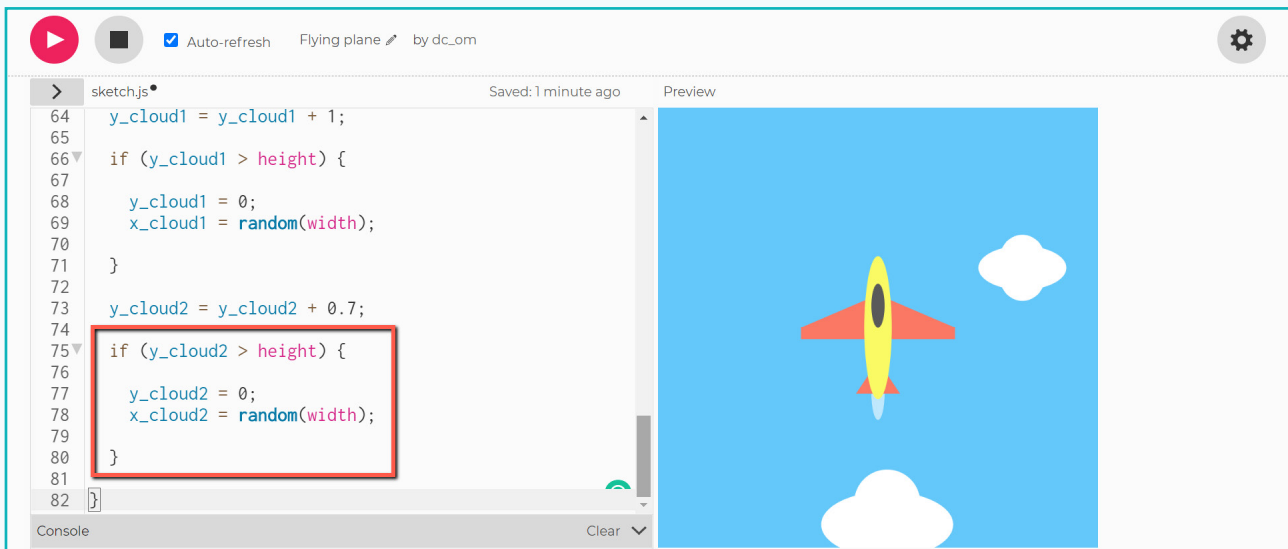
```

> sketch.js
57
58 //cloud1
59 fill(255);
60 ellipse(x_cloud1,y_cloud1,120,60);
61 ellipse(x_cloud1,y_cloud1-20,60);
62 ellipse(x_cloud1,y_cloud1+20,60);
63
64 y_cloud1 = y_cloud1 + 1;
65
66 if (y_cloud1 > height) {
67   y_cloud1 = 0;
68   x_cloud1 = random(width);
69 }
70
71
72
73 y_cloud2 = y_cloud2 + 0.7;
74
75
    
```

The plane seems to be moving above the second cloud that is moving slower than the first cloud.

Activity

- f. Add a similar if condition for cloud 2 to make it come back on the screen at a random x position from the upper edge.



When the code runs, we can watch the airplane flying through the sky!

In this lesson, we learnt about flowcharts that are pictorial representations of a step-by-step approach to solving a task. Following the steps from our flowchart, we created an animation of a plane flying through the sky.

Exercise

Answer in one line

A circle is drawn using a command `circle(loc_x, loc_y, 10)`; where `loc_x` and `loc_y` store the x and y position values of the centre of the circle. Width and height are default variables storing values of the width and height of the canvas respectively.

- 7 Write a single line of code such that if a circle goes out of the canvas from the right edge, it should enter again from the left edge.



- 8 Write a single line of code such that if a circle goes out of the canvas from the left edge, it should enter again from the right edge.



- 9 Write a single line of code such that if a circle goes out of the canvas from the top edge, it should enter again from the bottom edge.



Procedural animation is a method of creating computer graphics animation using code and algorithm.



Tech Flash

- **Flowchart** : A diagram that represents a step-by-step approach to solving a task.
- **Variable** : An entity or character that stores data in the form of numbers or words and is defined using the keyword 'var'.
- **If condition** : This statement executes a snippet of code only if a specified condition is true.
- **Increment** : An operation used to increase the value of a variable by a specific amount in each iteration.
- **Decrement** : An operation used to decrease the value of a variable by a specific amount in each iteration.